

- Automatic Image Analysis

Transformers

Simon Prince: Understanding Deep Learning, Chapter 12
Julia Turc: <https://www.youtube.com/watch?v=LE3NfEULV6k>

Natural language processing (NLP)

- Translation
- Question answering
- Summarizing
- Generating new text
- Correcting spelling and grammar
- Finding entities
- Classifying bodies of text
- Changing style etc.

Transformers

- Motivation
- Dot-product self-attention
- Matrix form
- The transformer
- NLP pipeline

Natural Language Processing (NLP)

Example Task: Sentiment Classification

Input: review of
TV show



Machine
learning
model



Output: class

positive 
negative 

Text Encoding

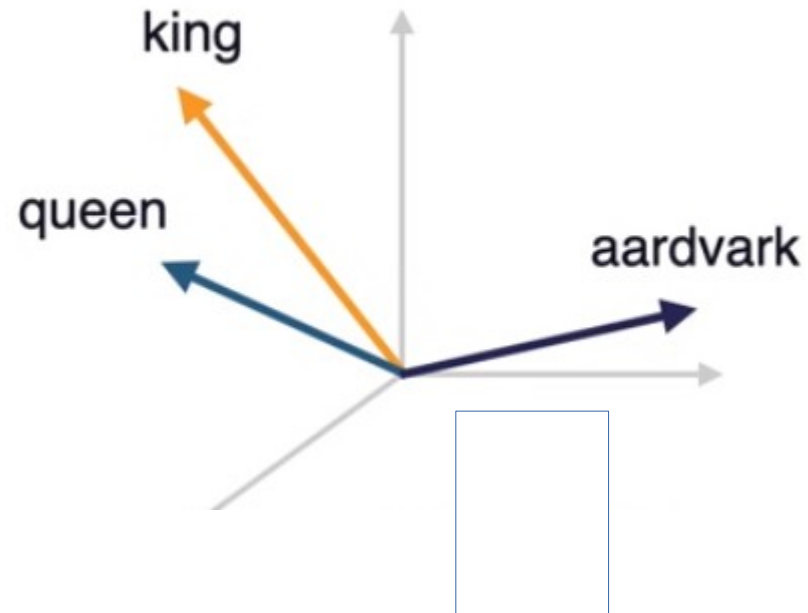
Naïve view: words are **discrete** and **independent** tokens.

1. Build a dictionary of tokens
2. Assign an index to each token
3. Use the “one-hot” embedding of the tokens

Token	Index	One-hot vector
aardvark	0	[1, 0, 0, ...]
...		
king	123	[0, ..., 0, 1, 0, ...]
queen	124	[0, ..., 0, 0, 1, ...]
...		

Text Encoding

Token	Index	One-hot vector	Continuous vector
aardvark	0	[1, 0, 0]	[0.3, 1.9, -0.4]
king	1	[0, 1, 0]	[2.1, -0.7, 0.2]
queen	2	[0, 0, 1]	[0.5, 1.3, 0.9]

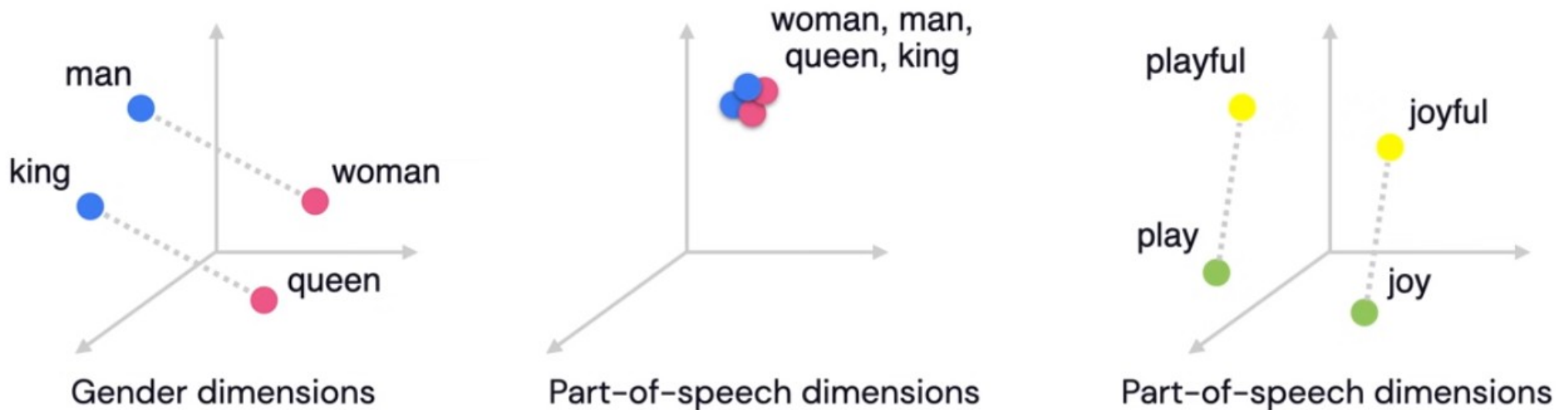


Desirable Word Embedding Properties

- Embeddings can encode multiple aspects of a token
token queen = [0.1, -0.3, 1.2, -0.4, 0.02, 1.1, -0.25, ...]

Gender Part-of-speech

- The mechanism to express relationships is relative vector distance



Motivation

Design neural network to encode and process text:

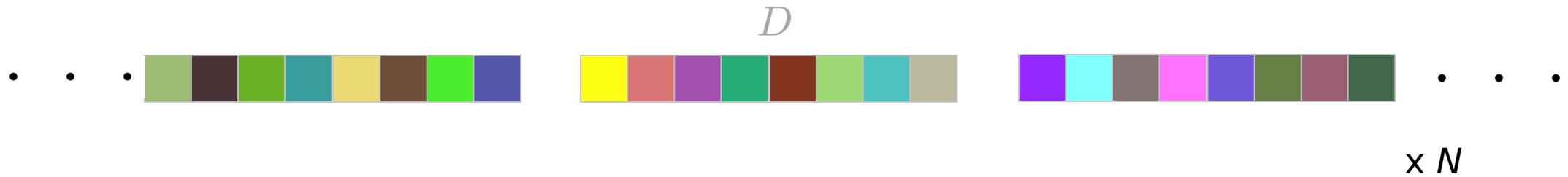
The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.



Motivation

Design neural network to encode and process text:

The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.



- Each of the 37 words might be represented by an embedding vector of length 1024.
- The network should share parameters across different positions.
- The word „it“ should be connected to the word „restaurant“.
- „It“ should „pay **attention**“ to „restaurant“.

Motivation

Design neural network to encode and process text:

The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.

The word **their** must “attend to” the word **restaurant**.

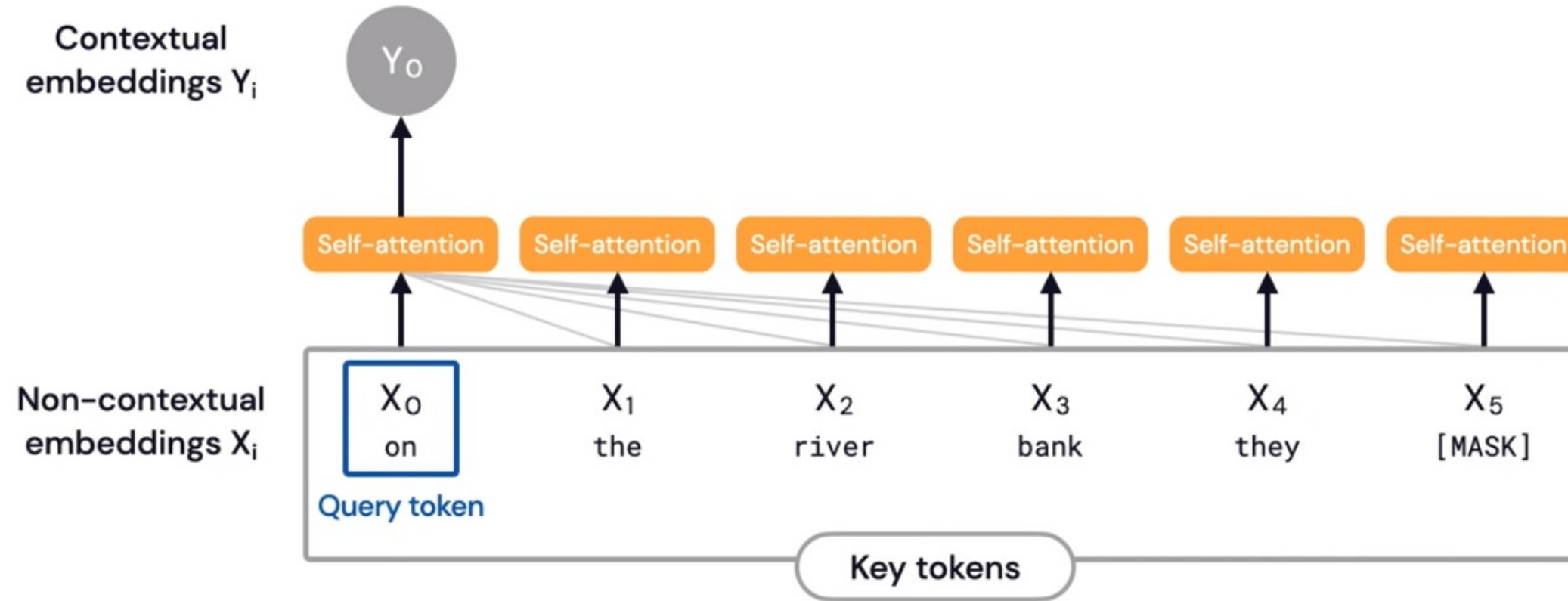
Conclusions:

- There must be connections between the words.
- The strength of these connections will depend on the words themselves.

Transformers

- Motivation
- Dot-product self-attention
- Matrix form
- The transformer
- NLP pipeline

Self-Attention

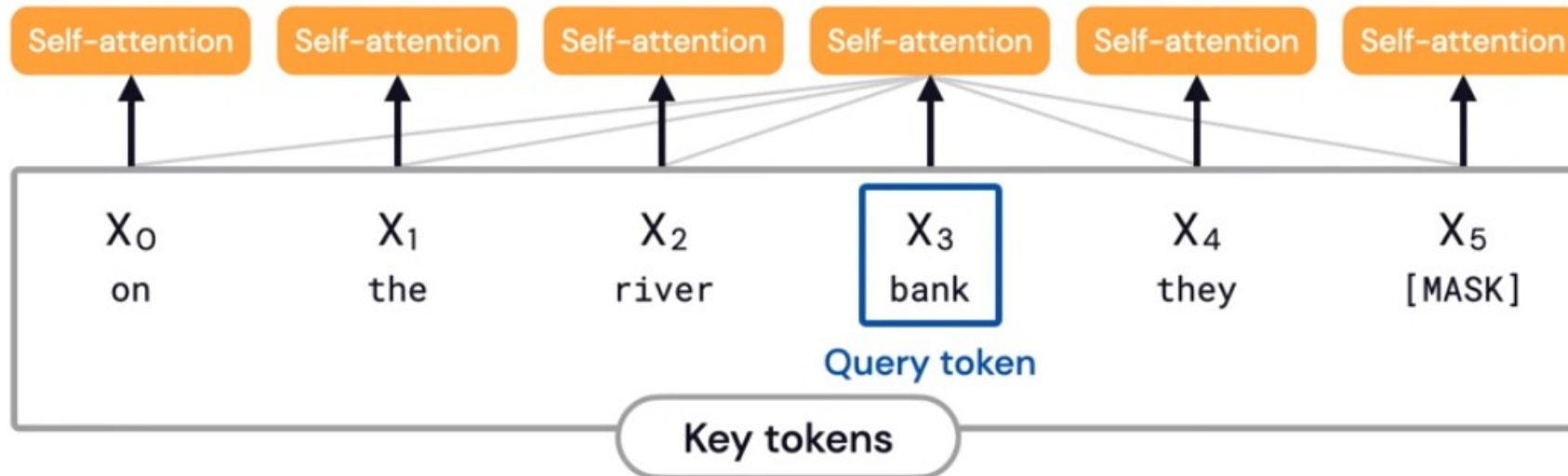


Self-Attention

Attention is similar to a **soft** dictionary lookup.

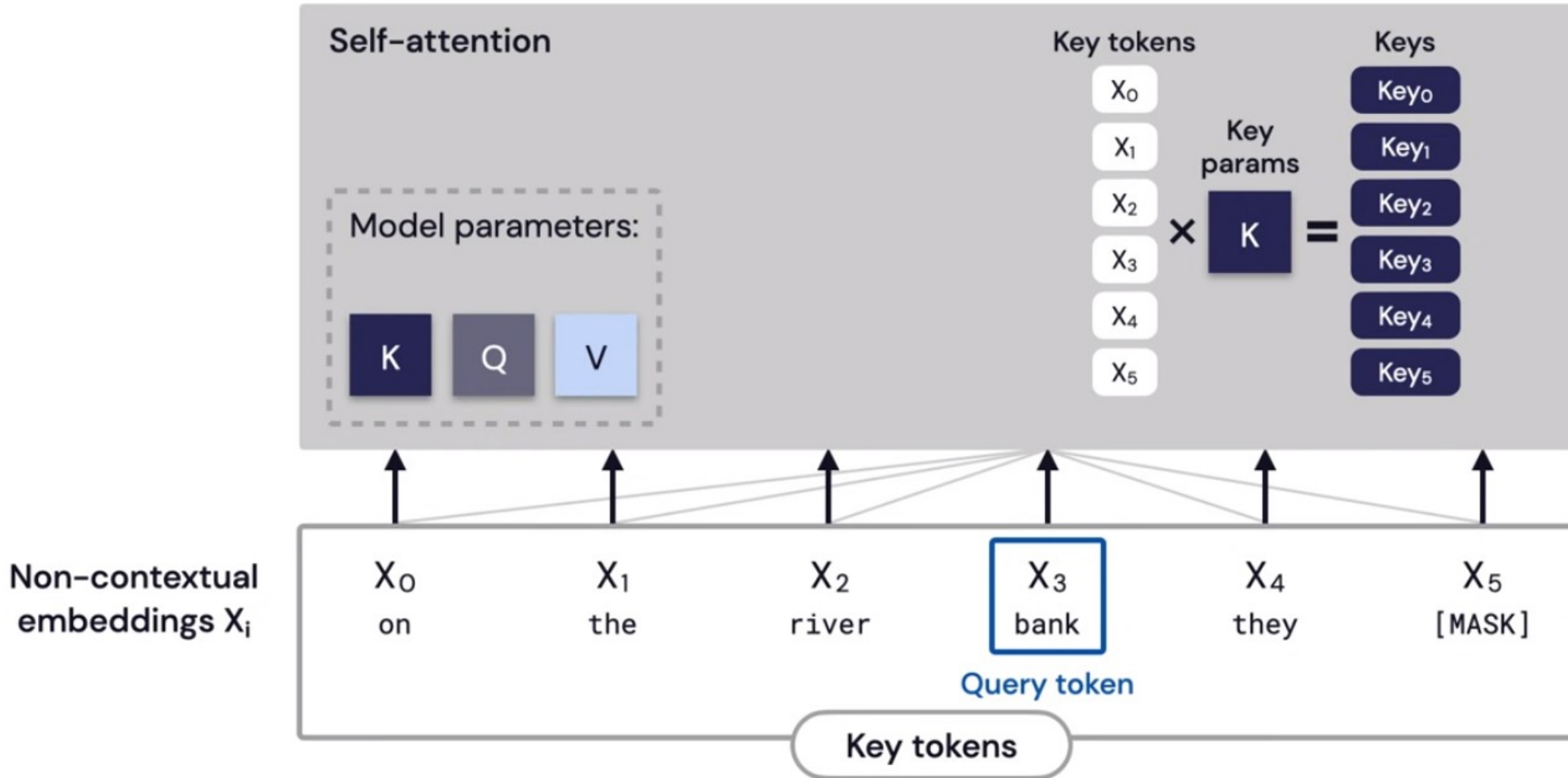
dict = {	lookup(dict, bank) =
on: v0,	v0 * 0.04 +
the: v1,	v1 * 0.04 +
river: v2,	v2 * 0.30 +
bank: v3,	v3 * 0.60 +
they: v4,	v4 * 0.02 +
[MASK]: v5, }	v5 * 0.0

Contextual
embeddings Y_i

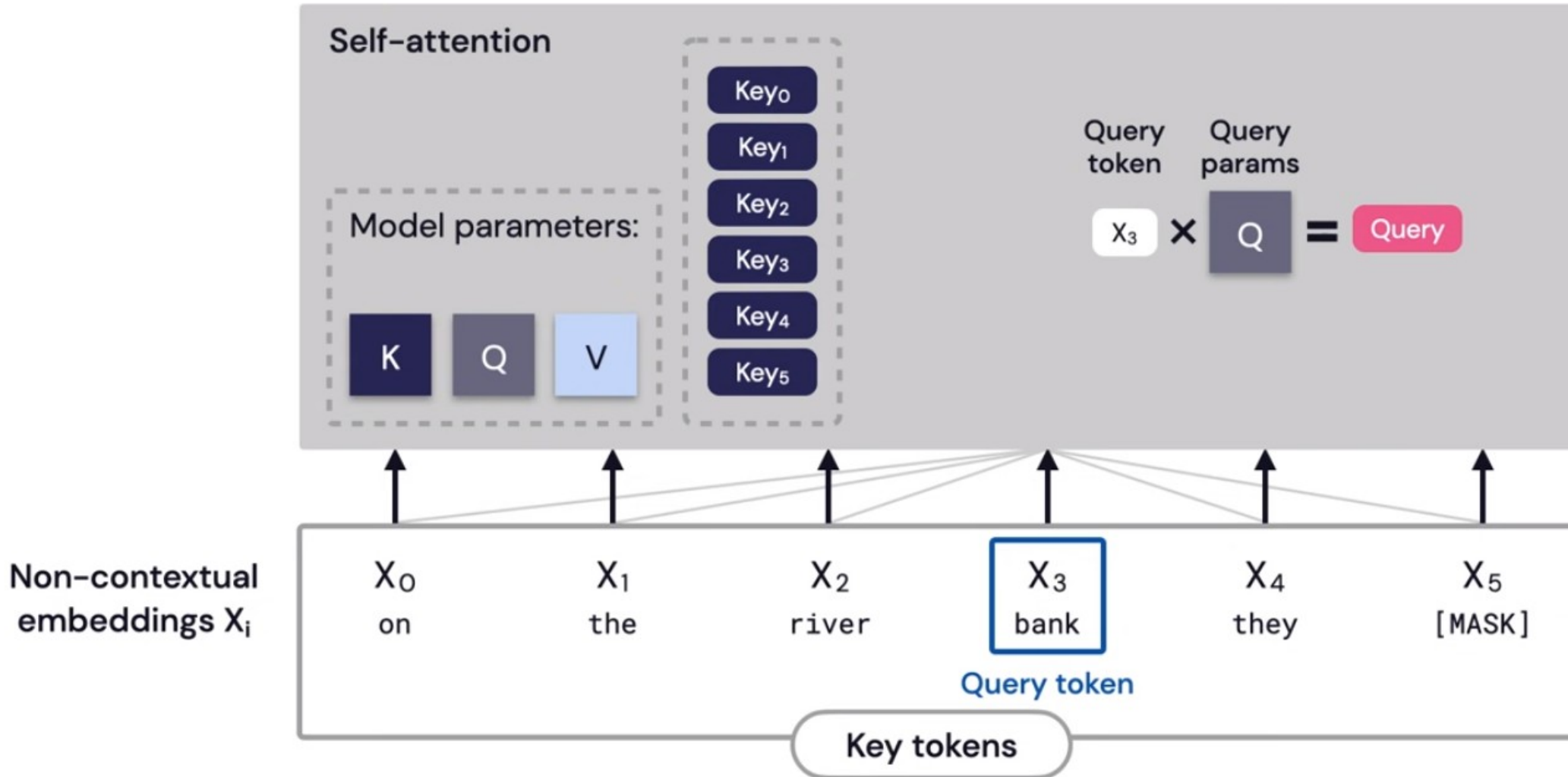


Non-contextual
embeddings X_i

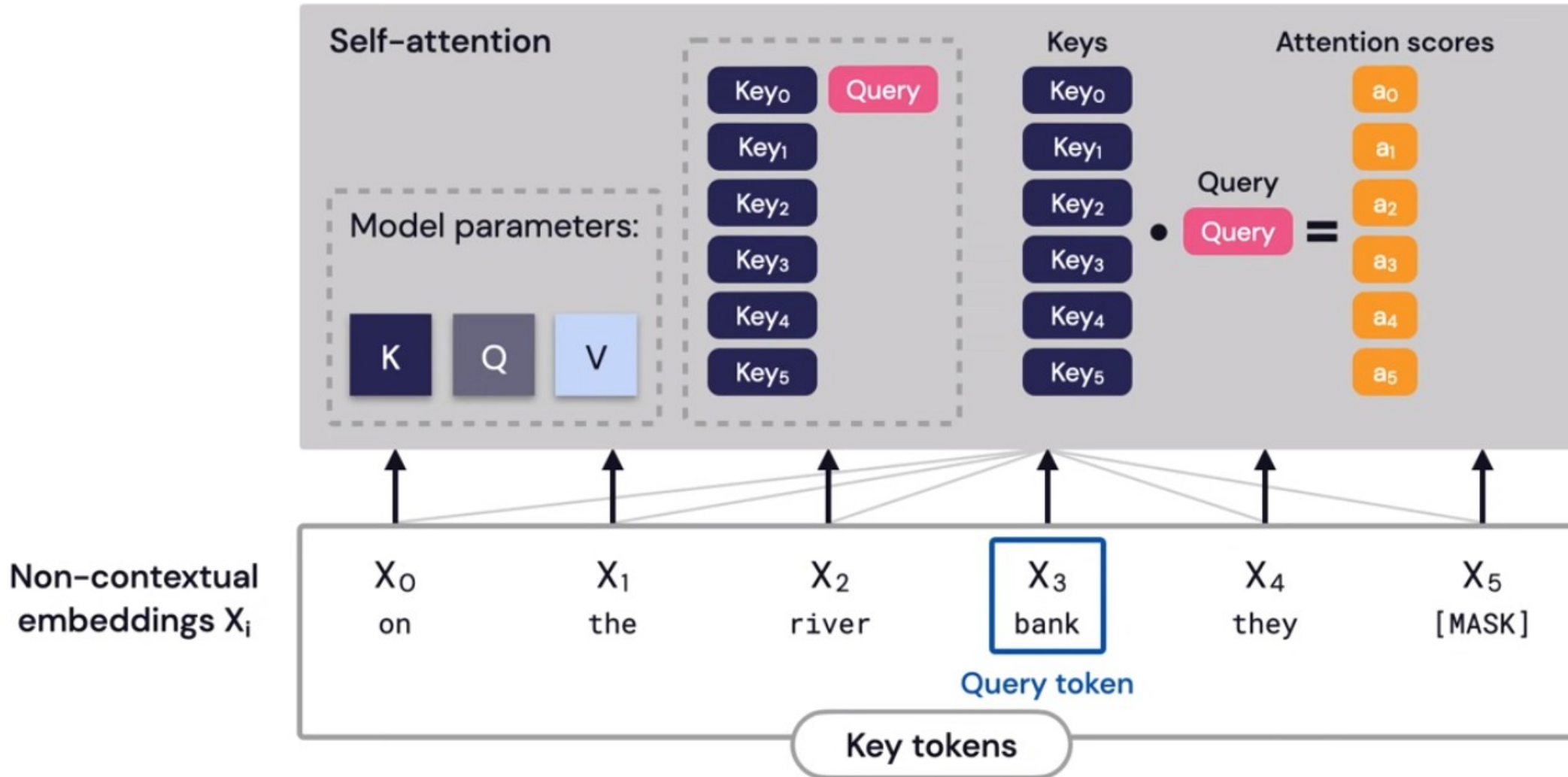
Self-Attention



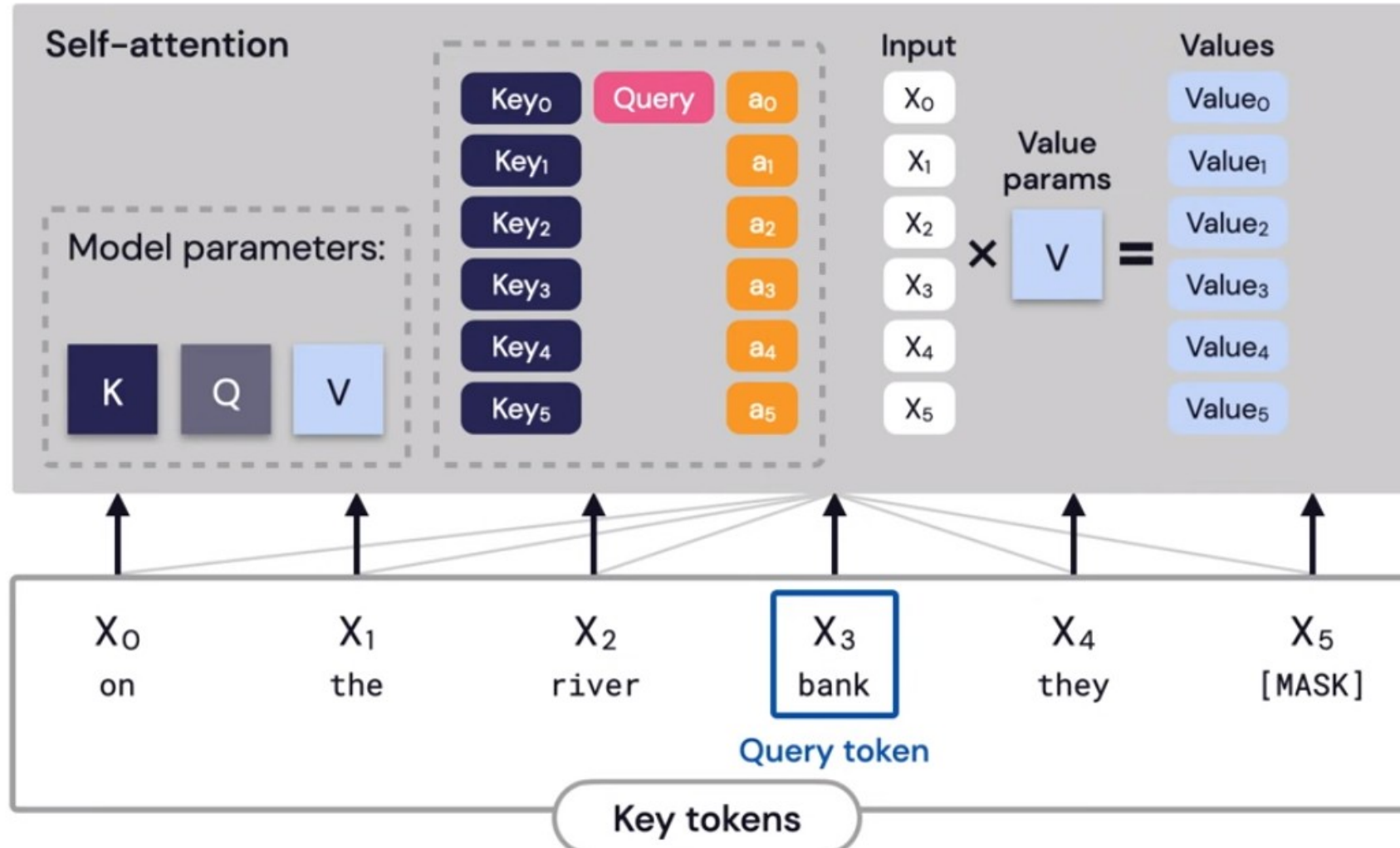
Self-Attention



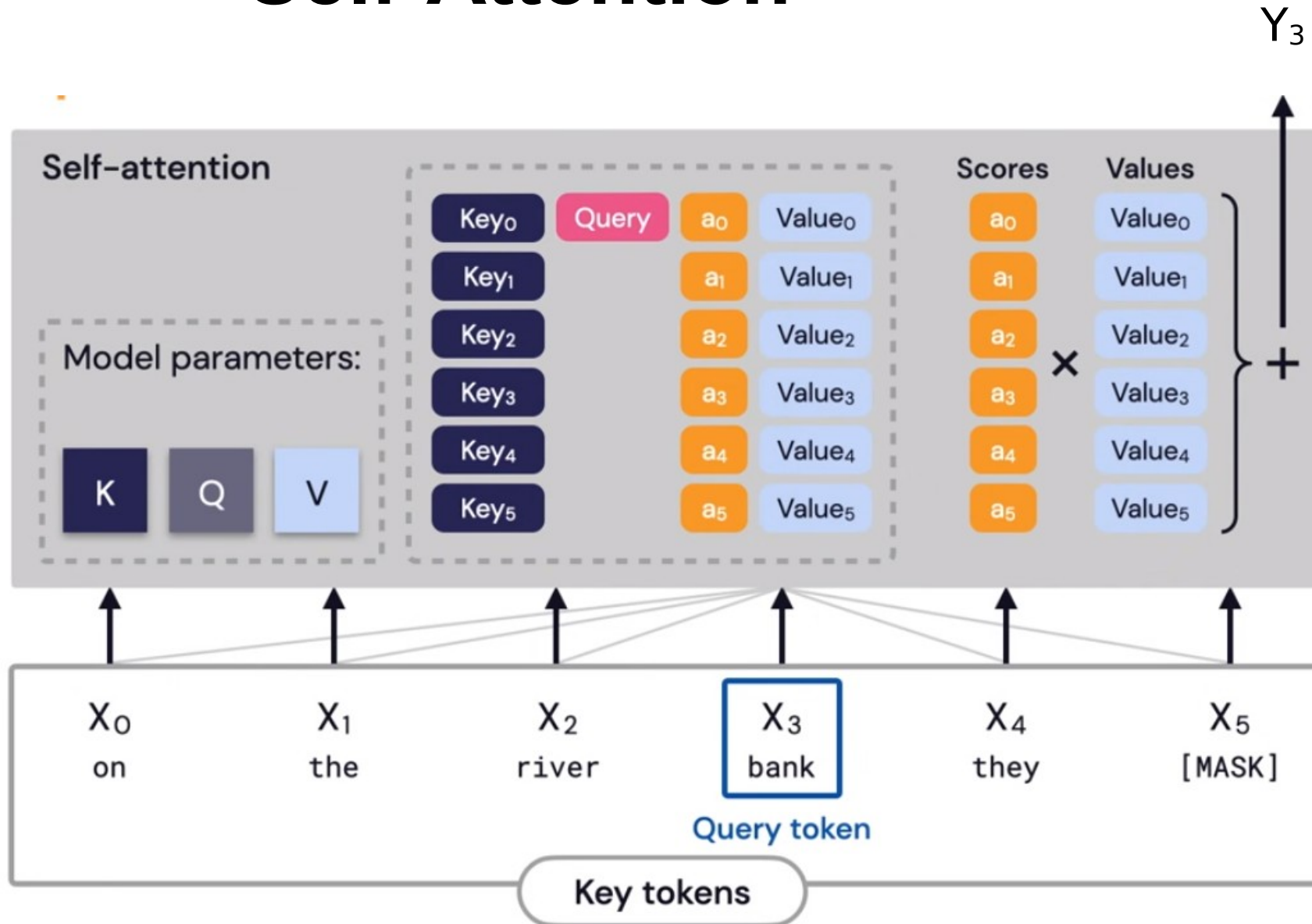
Self-Attention



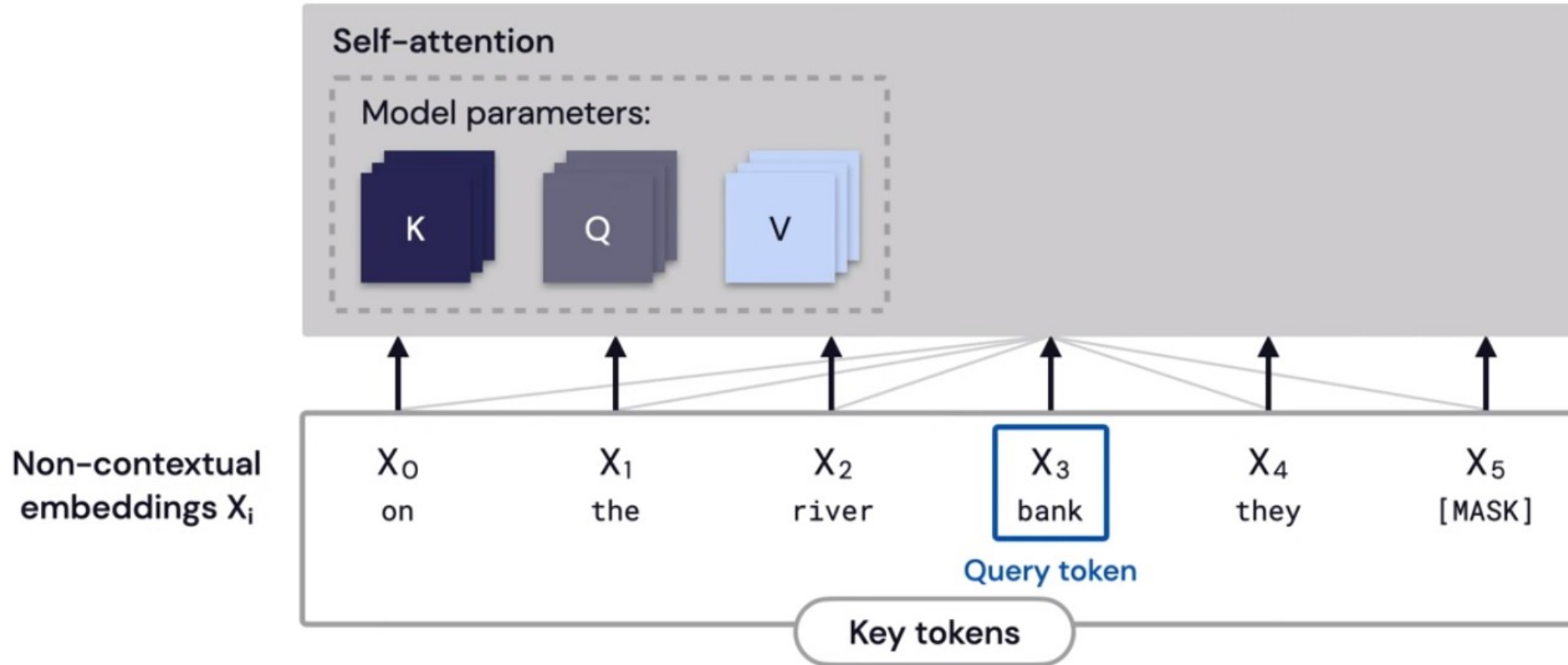
Self-Attention



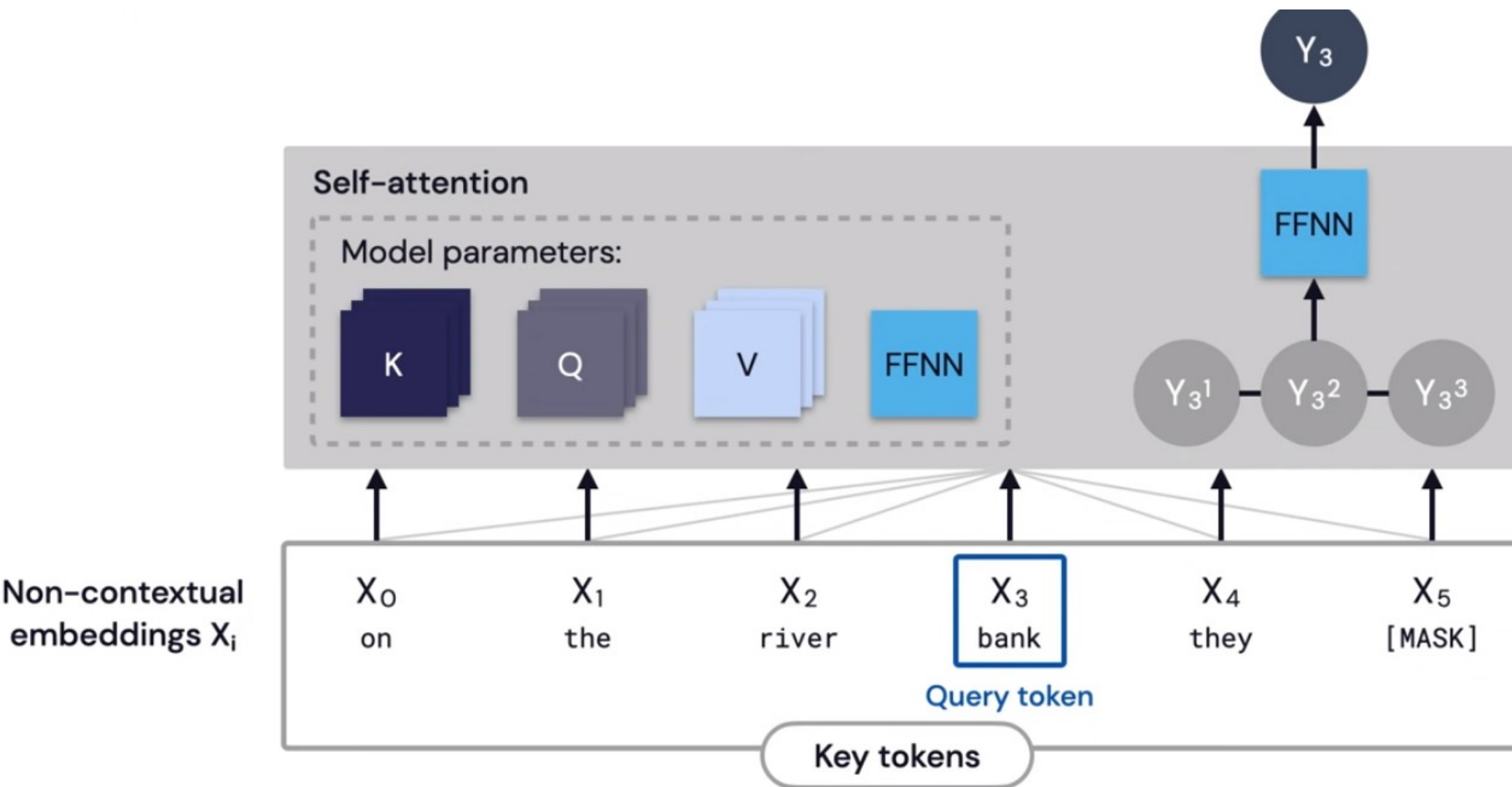
Self-Attention



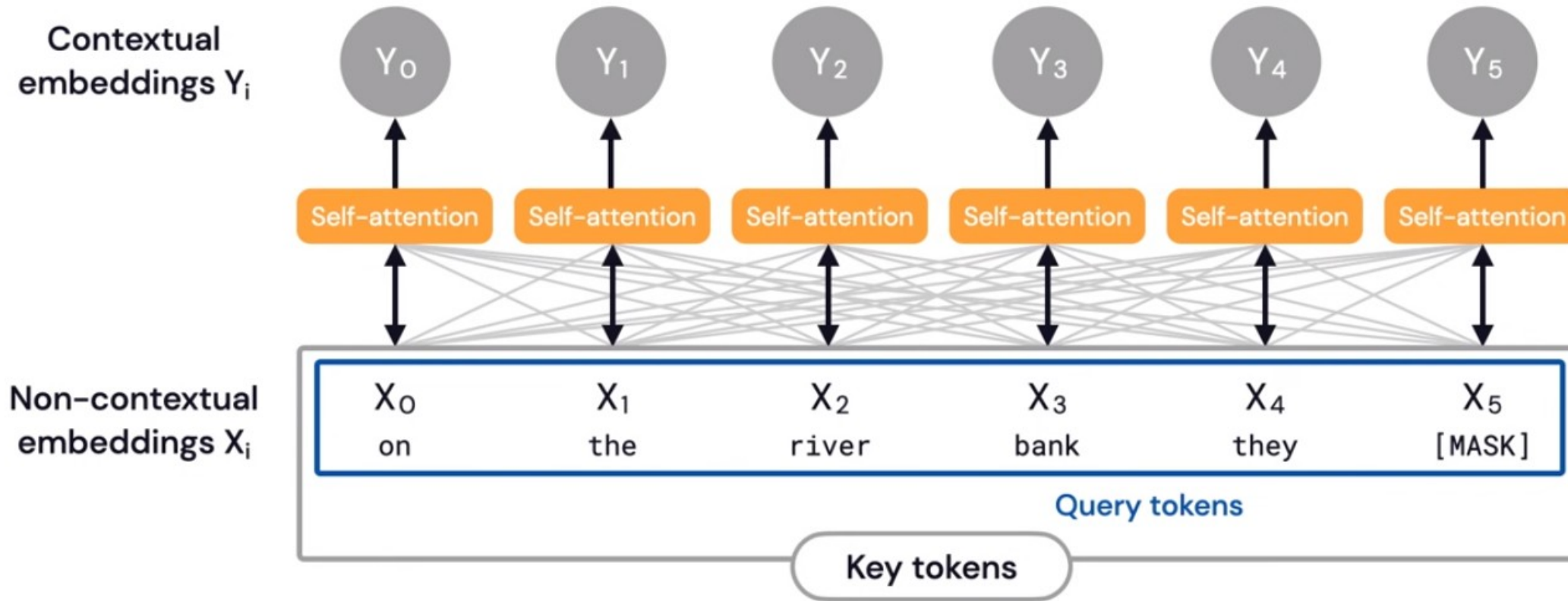
Multi-Head Self-Attention



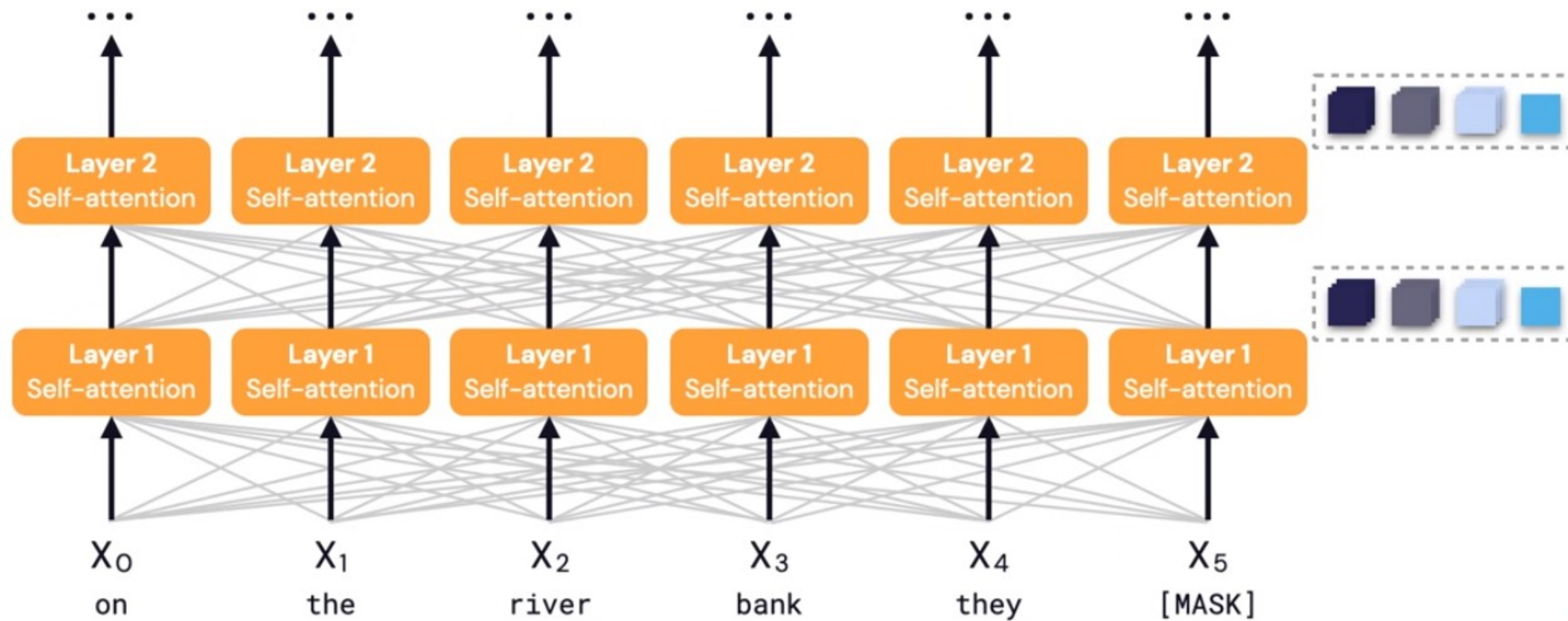
Attention Heads



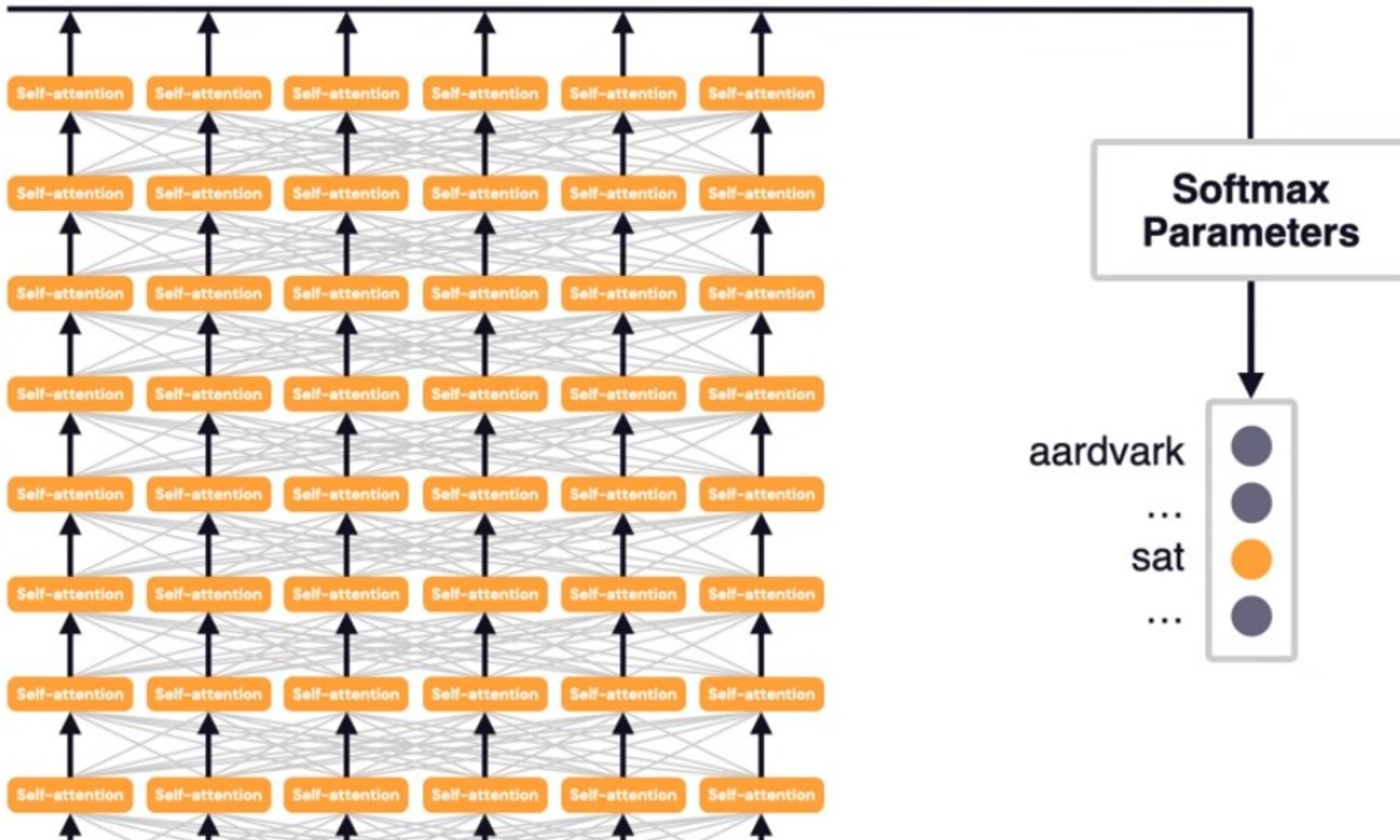
$O(n^2)$ complexity, but in parallel



The Transformer Encoder



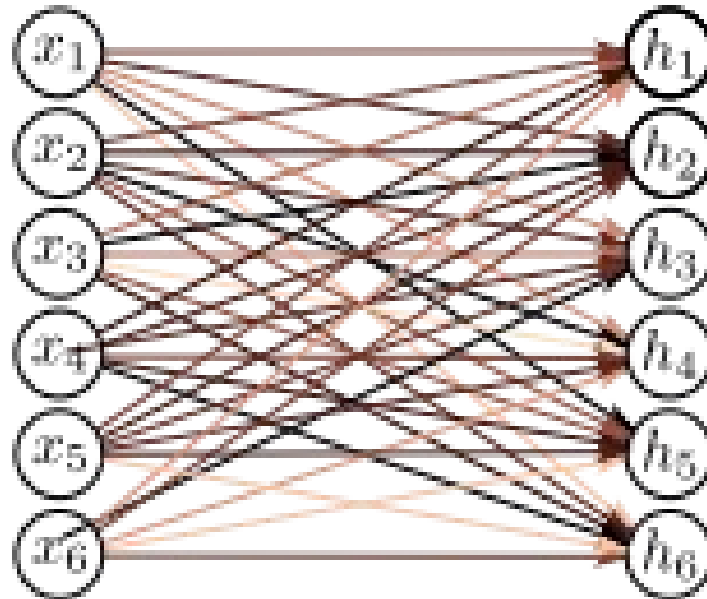
The Transformer Encoder



Dot-product self attention

- Requirements for text processing:
 - Use **parameter sharing** to allow processing text of varying lengths
 - Contain **connections between** words depending on **individual words**
- Transformers do that by using dot-product self-attention.
- We know standard fully-connected layers:

$$\mathbf{h} = \mathbf{a}[\boldsymbol{\beta} + \boldsymbol{\Omega}\mathbf{x}]$$



Dot-product self attention

- Takes N inputs of size $D \times 1$ and returns N outputs of size $D \times 1$
- Computes N **values** (no ReLU)

$$\mathbf{v}_n = \beta_v + \Omega_v \mathbf{x}_n$$

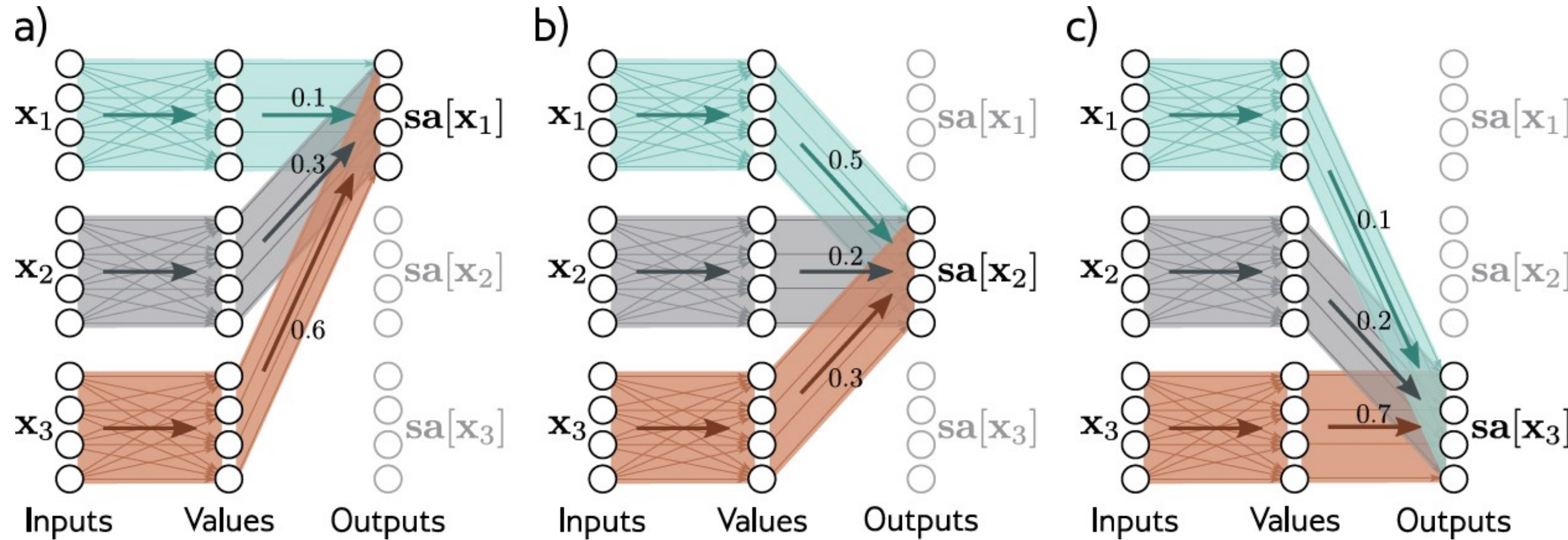
Value computation can be viewed as a sparse matrix operation with shared parameters.

- N outputs are weighted sums of these values

$$\mathbf{sa}[\mathbf{x}_n] = \sum_{m=1}^N a[\mathbf{x}_n, \mathbf{x}_m] \mathbf{v}_m$$

- Scalar weights **a** depend on the inputs themselves:
attention that the n^{th} output pays to the m^{th} input

Self-attention as routing



- Each output can be thought of as a different routing of the N values.

Computing attention weights

- Compute N “queries” and N “keys” from input

$$\mathbf{q}_n = \beta_q + \Omega_q \mathbf{x}_n$$

$$\mathbf{k}_n = \beta_k + \Omega_k \mathbf{x}_n,$$

Like in information theory: “query” (term in input sequence) is compared with “keys” (all other terms in input sequence).

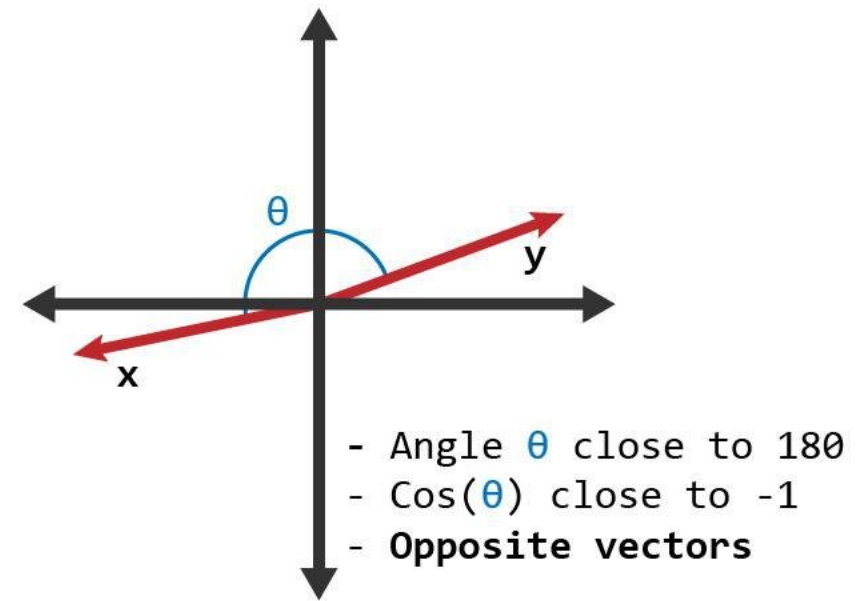
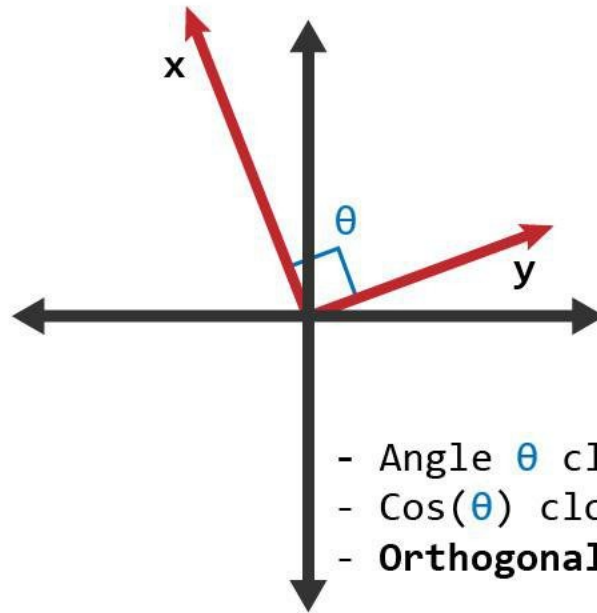
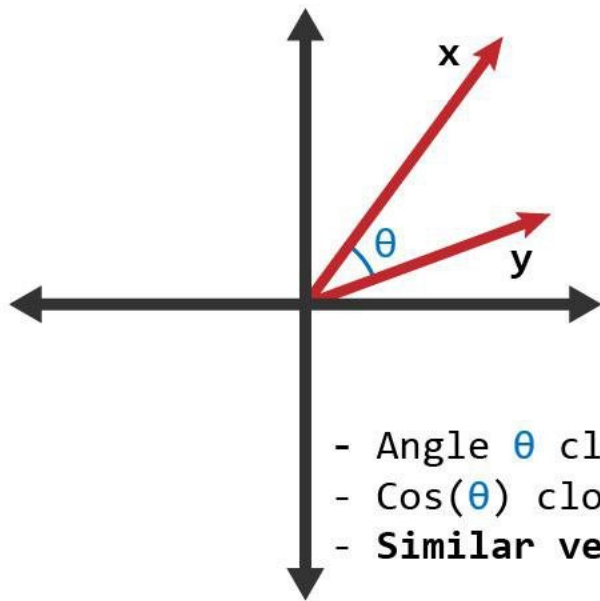
- Calculate similarity (dot product) and pass through softmax:

$$\begin{aligned} a[\mathbf{x}_n, \mathbf{x}_m] &= \text{softmax}_m [\text{sim}[\mathbf{k}_m \mathbf{q}_n]] \\ &= \frac{\exp [\text{sim}[\mathbf{k}_m \mathbf{q}_n]]}{\sum_{m'=1}^N \exp [\text{sim}[\mathbf{k}'_m \mathbf{q}_n]]}, \end{aligned}$$

- Attention weight (dot-product operation) return a **measure of similarity** between its inputs.

Dot product = measure of similarity

$$\mathbf{x}^T \mathbf{y} = |\mathbf{x}| \cdot |\mathbf{y}| \cdot \cos(\theta)$$



Summary - and why/how the **Query-Key-Value + scaled-dot-product** pattern works

- Queries, Keys and Values are **linear projections** of the input.
- Self-attention weights **a** are a softmax-function of **dot-product similarities** of the query and keys.
- Three **distinct subspaces** are learned:
 - **q[x]**: what I want (How to formulate a search request.)
 - **K[x]**: how to index myself (How to index an entry.)
 - **v[x]**: what I can provide (What to store at that index.)
- Query $\Leftrightarrow$ search request, Key $\Leftrightarrow$ index entries, Value $\Leftrightarrow$ payload. The model performs a **learned “content-based lookup”**.
- The scaled dot product is a learnable bilinear kernel that gives **power to approximate**.
- In the forward path it looks like there is no relation between value parameters on the one hand and query parameters and key parameters on the other hand.
- But as all three are trained together, i.e. in the backward path, there is a **tight coupling through a shared loss**.
- If we freeze Ω_v after random initialization, but continue training Ω_q and Ω_k , performance collapses $\rightarrow$ values are not arbitrary.

Summary - and why/how the **Query-Key-Value + scaled-dot-product** pattern works

- Ω_k is learned to encode **how this token should be found** (e.g. part of speech, positional cue)
so that attention can match queries that need that kind of information
- Ω_q is learned to encode what this token needs to know (**e.g. “find the verb I depend on”**)
so that attention can point probability mass toward the right keys
- Ω_v is learned to encode **what this token will contribute** once found (e.g., its semantic embedding)
so that attention can provide useful content once the address is chosen

Summary - and why/how the **Query-Key-Value + scaled-dot-product** pattern works

- Balances expressiveness vs. compute efficiency
- Variants have been tried, but have rarely been better, e.g.
 - Cosine or Euclidean similarities
 - $\Omega_k = \Omega_q = \Omega_v$: converges but under-fits

Motivation

Design neural network to encode and process text:

The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.

Conclusions:

- We need a model where parameters don't increase with input length

$$\phi = \{\beta_v, \Omega_v, \beta_q, \Omega_q, \beta_k, \Omega_k\}$$

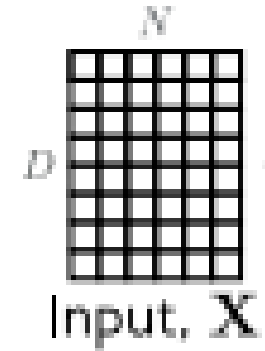
- There must be connections between the words.
- The strength of these connections will depend on the words themselves.

Transformers

- Motivation
- Dot-product self-attention
- Matrix form
- The transformer
- NLP pipeline

Matrix form

- Store N input vectors in matrix X



- Compute values, queries and keys:

$$\mathbf{V}[\mathbf{X}] = \beta_v \mathbf{1}^T + \Omega_v \mathbf{X}$$

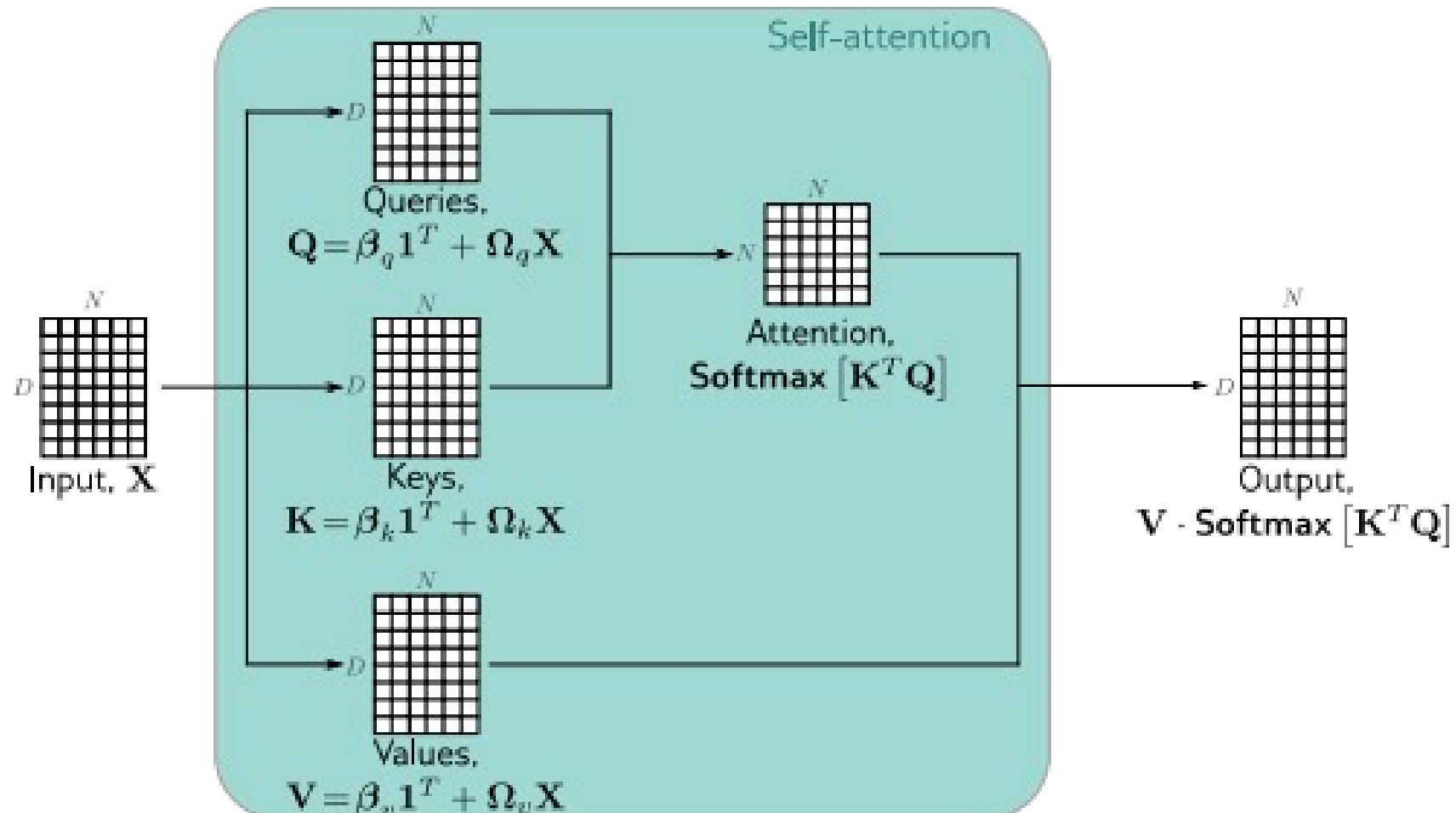
$$\mathbf{Q}[\mathbf{X}] = \beta_q \mathbf{1}^T + \Omega_q \mathbf{X}$$

$$\mathbf{K}[\mathbf{X}] = \beta_k \mathbf{1}^T + \Omega_k \mathbf{X},$$

- Combine self-attentions

$$\mathbf{Sa}[\mathbf{X}] = \mathbf{V}[\mathbf{X}] \cdot \text{Softmax} \left[\mathbf{K}[\mathbf{X}]^T \mathbf{Q}[\mathbf{X}] \right]$$

Matrix form



Position encoding

- Self-attention is equivariant to permuting word order
- But word order is important in language:

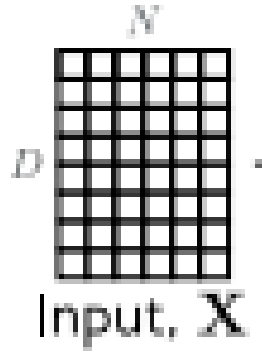
The man ate the fish

vs.

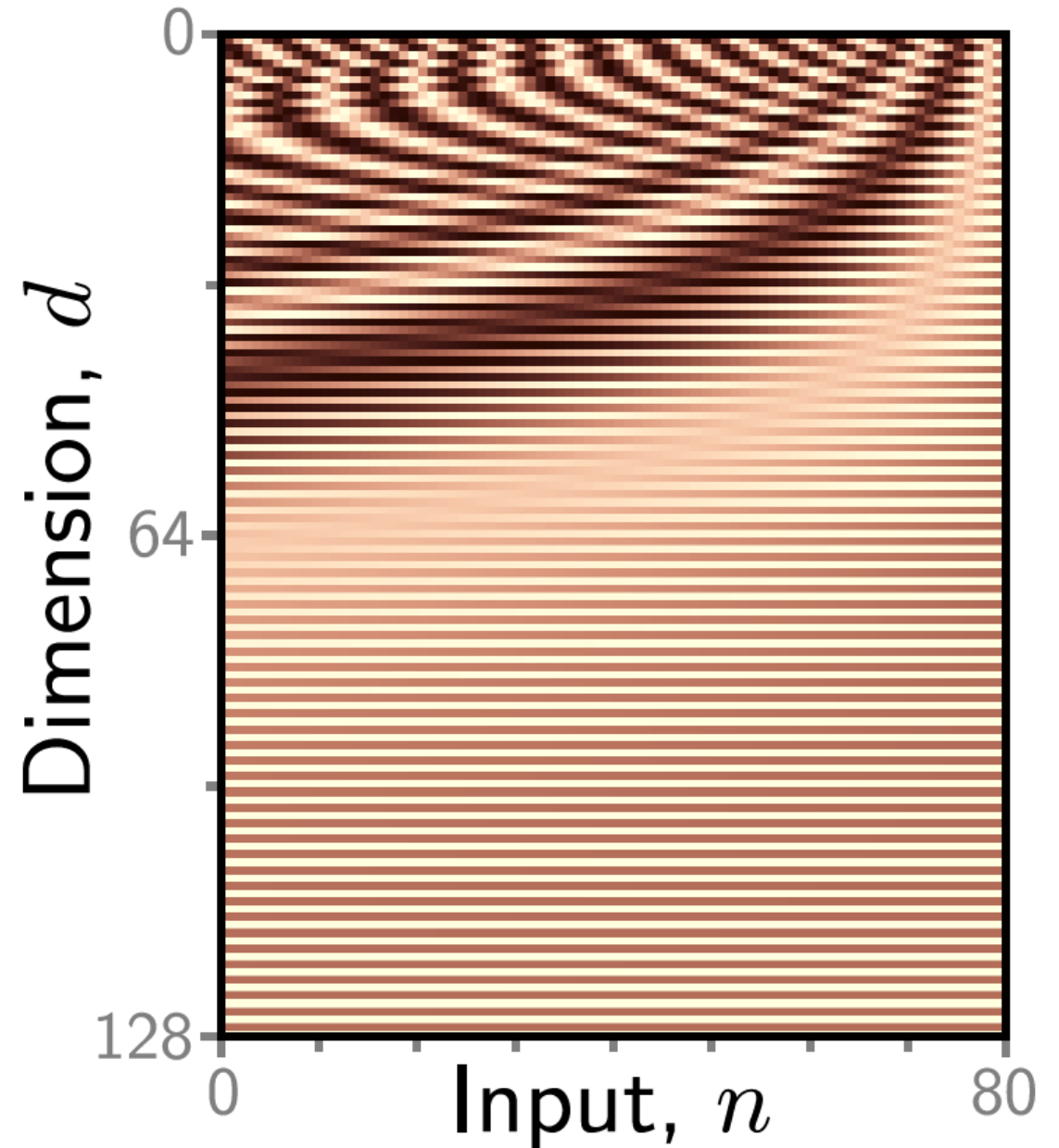
The fish ate the man

- Order is important also in image processing.

Position encoding

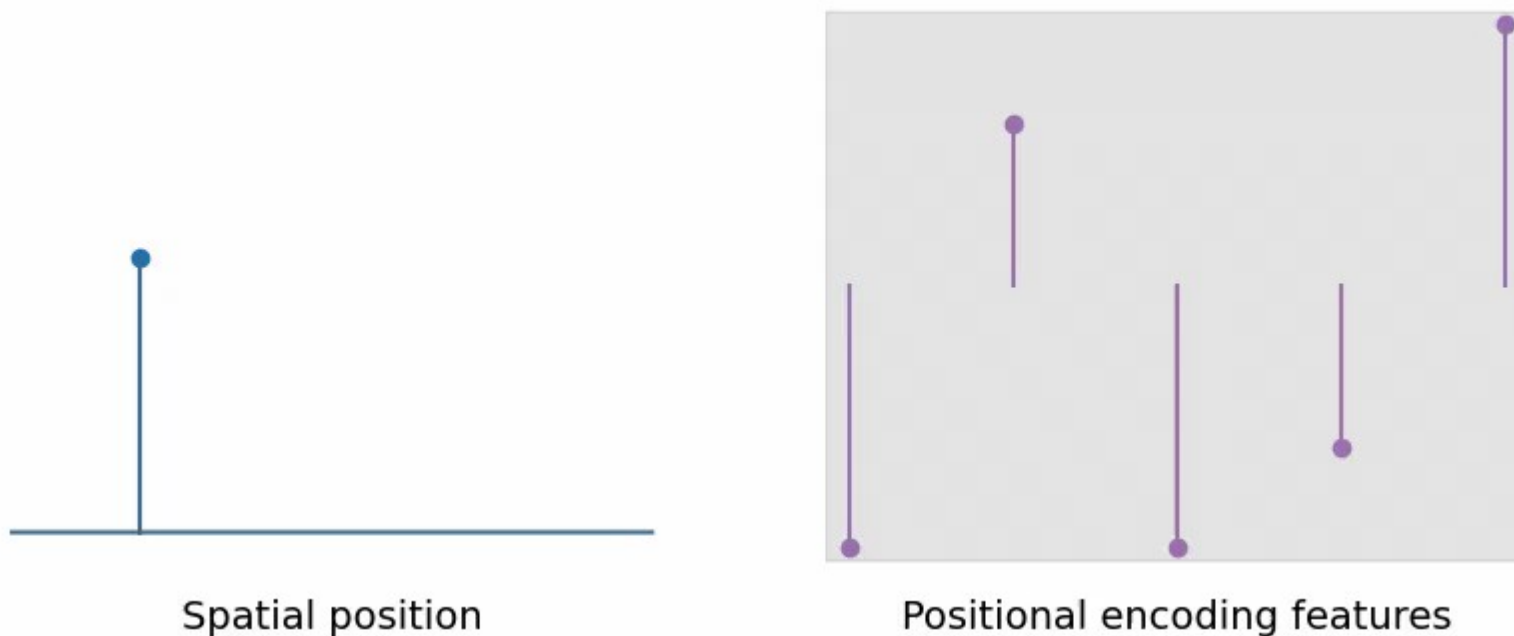


- Here positional encoding are predefined **sinusoidal** patterns.
- The encoding matrix π can be **added** to the network inputs X .



Positional Encoding

- Single point in space is mapped to a feature vector, where each element is generated by a sinusoid with an exponentially increasing frequency



$$\gamma_{\omega}(x) = \sin(\omega x), \quad \gamma(x) = \left(\gamma_{2^{\ell}}(x) \right)_{\ell=0}^{L-1}$$

Absolute and Relative Positional Encoding

- Positional encoding can be **absolute**.
- Then the matrix π can be added to $\mathbf{X}$.
- Positional encoding can also be **relative**.
- Then a parameter for each offset is used to modify the attention matrix by adding these.

$$\mathbf{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}[\mathbf{K}^T \mathbf{Q}]$$



$$\mathbf{Sa}[\mathbf{X}] = (\mathbf{V} + \mathbf{\Pi}) \cdot \text{Softmax}[(\mathbf{K} + \mathbf{\Pi})^T (\mathbf{Q} + \mathbf{\Pi})]$$

Multiple Heads

- Multiple self-attention mechanisms are usually applied in parallel.

$$\mathbf{V}_h = \beta_{vh} \mathbf{1}^T + \Omega_{vh} \mathbf{X}$$

$$\mathbf{Q}_h = \beta_{qh} \mathbf{1}^T + \Omega_{qh} \mathbf{X}$$

$$\mathbf{K}_h = \beta_{kh} \mathbf{1}^T + \Omega_{kh} \mathbf{X}$$

- h^{th} self-attention mechanism (head):

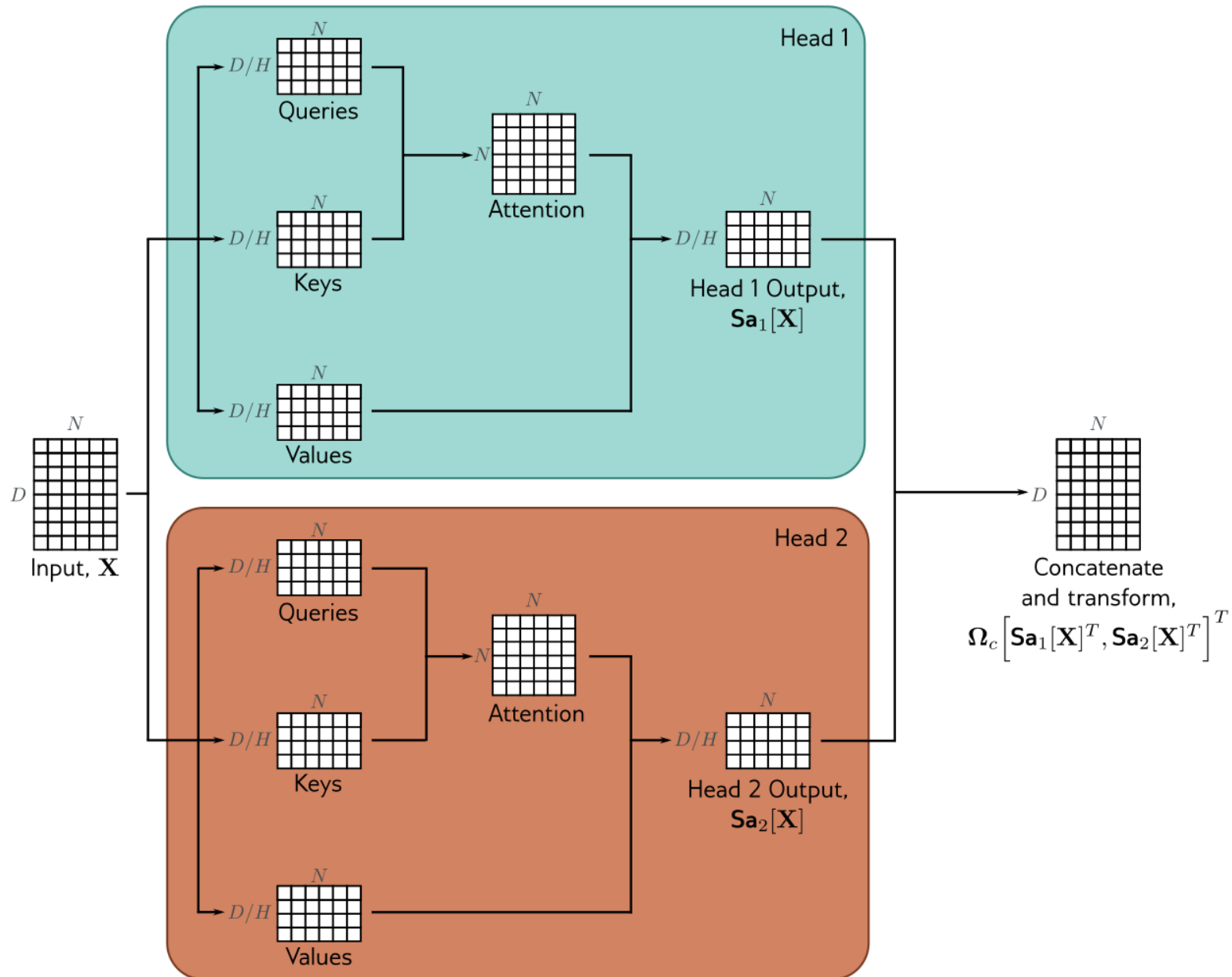
$$\mathbf{Sa}_h[\mathbf{X}] = \mathbf{V}_h \cdot \text{Softmax} \left[\frac{\mathbf{K}_h^T \mathbf{Q}_h}{\sqrt{D_q}} \right]$$

- Usually, outputs of heads are vertically concatenated, and another linear transform is applied

$$\mathbf{MhSa}[\mathbf{X}] = \Omega_c \left[\mathbf{Sa}_1[\mathbf{X}]^T, \mathbf{Sa}_2[\mathbf{X}]^T, \dots, \mathbf{Sa}_H[\mathbf{X}]^T \right]^T$$

- Seems to be necessary to make self-attention more robust to bad initializations.

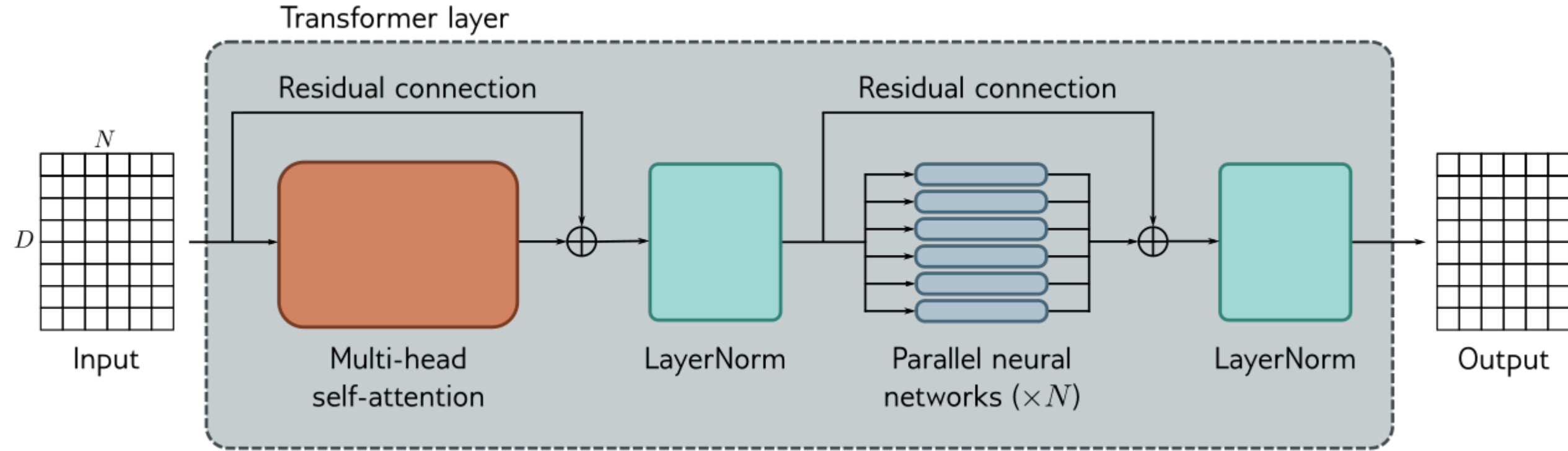
Multi-Head Self-Attention



Transformers

- Motivation
- Dot-product self-attention
- Matrix form
- The transformer
- NLP pipeline

Transformer Layers



- Residual connections around multi-head self-attention
- LayerNorm uses statistics across the tokens
- 2nd residual layer where the same fully-connected layer is applied separately to each word.

Transformer Layers

- Matrix equations

$$\mathbf{X} \leftarrow \mathbf{X} + \text{MhSa}[\mathbf{X}]$$

$$\mathbf{X} \leftarrow \text{LayerNorm}[\mathbf{X}]$$

$$\mathbf{x}_n \leftarrow \mathbf{x}_n + \text{mlp}[\mathbf{x}_n]$$

$$\mathbf{X} \leftarrow \text{LayerNorm}[\mathbf{X}]$$

- In a real network, the data is passed through a series of these transformer layers.

Transformers

- Motivation
- Dot-product self-attention
- Matrix form
- The transformer
- NLP pipeline

Transformers for Natural Language Processing

- 1) Tokenizer splits text into words or word fragments.
- 2) Each token is mapped to a learned embedding.
- 3) Embeddings are passed through series of transformer layers.

Tokenizer

Goal: **Tokenizer** chooses input “units”

- Inevitably, some words (e.g., names) will not be in the vocabulary.
- It's not clear how to handle punctuation
- The vocabulary would need different tokens for versions of the same word with different suffixes (e.g., walk, walks, walked, walking) and there is no way to clarify that these variations are related

Solution: **Sub-word tokenization**

a) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	s	a	t	o	h		u	b	d	w	c	f	i	m	n	p	r
33	28	15	12	11	8	6	6	4	3	3	3	2	1	1	1	1	1	1

b) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	se	a	t	o	h		u	b	d	w	c	s	f	i	m	n	p	r
33	15	13	12	11	8	6	6	4	3	3	3	2	2	1	1	1	1	1	1

c) a_sailor_went_to_sea_sea_sea_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

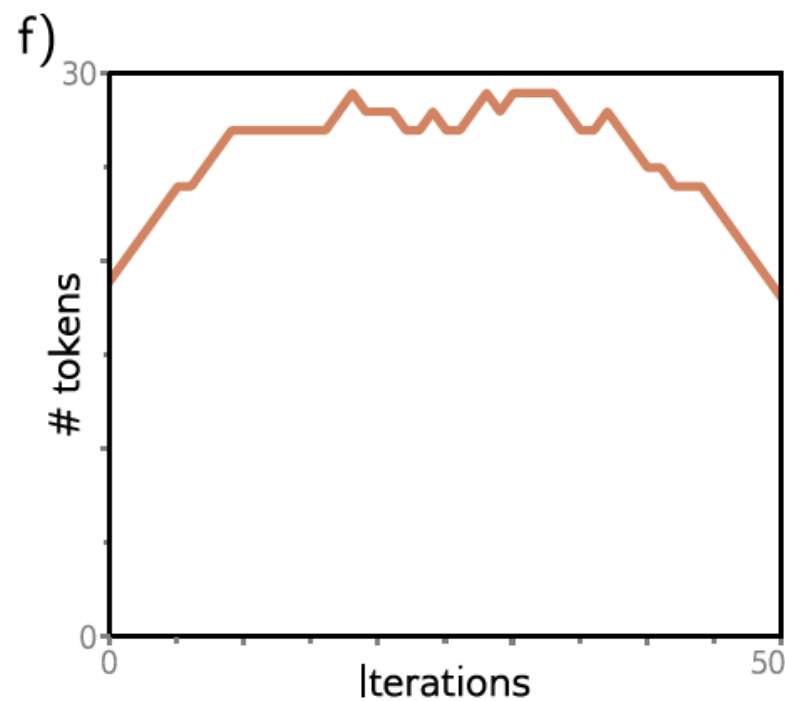
_	se	a	e	t	o	h		u	b	d	e	w	c	s	f	i	m	n	p	r
21	13	12	12	11	8	6	6	4	3	3	3	3	2	2	1	1	1	1	1	1

⋮ ⋮

d) see_sea_e_b|w_a_could_hat_he_o_t_t_the_to_u_a_d_f_m_n_p_s_sailor_to
7 6 4 3 3 3 3 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1

⋮ ⋮ ⋮

e) see_sea_could_he_the_a_all_blue_bottom_but_deep_of_sailor_that_to_was_went_what_
7 6 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1



- Initially, tokens are just characters.
- Most commonly occurring pair: “se”
- Then “e_” are merged.
- After 22 iterations mix of letters, word fragments and words

a) a_sailor_went_to_sea_sea_sea_
 to_see_what_he_could_see_see_see_
 but_all_that_he_could_see_see_see_
 was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	s	a	t	o	h	l	u	b	d	w	c	f	i	m	n	p	r
33	28	15	12	11	8	6	6	4	3	3	3	2	1	1	1	1	1	1

b) a_sailor_went_to_sea_sea_sea_
 to_see_what_he_could_see_see_see_
 but_all_that_he_could_see_see_see_
 was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	se	a	t	o	h	l	u	b	d	w	c	s	f	i	m	n	p	r
33	15	13	12	11	8	6	6	4	3	3	3	2	2	1	1	1	1	1	1

c) a_sailor_went_to_sea_sea_sea_
 to_see_what_he_could_see_see_see_
 but_all_that_he_could_see_see_see_
 was_the_bottom_of_the_deep_blue_sea_sea_sea_

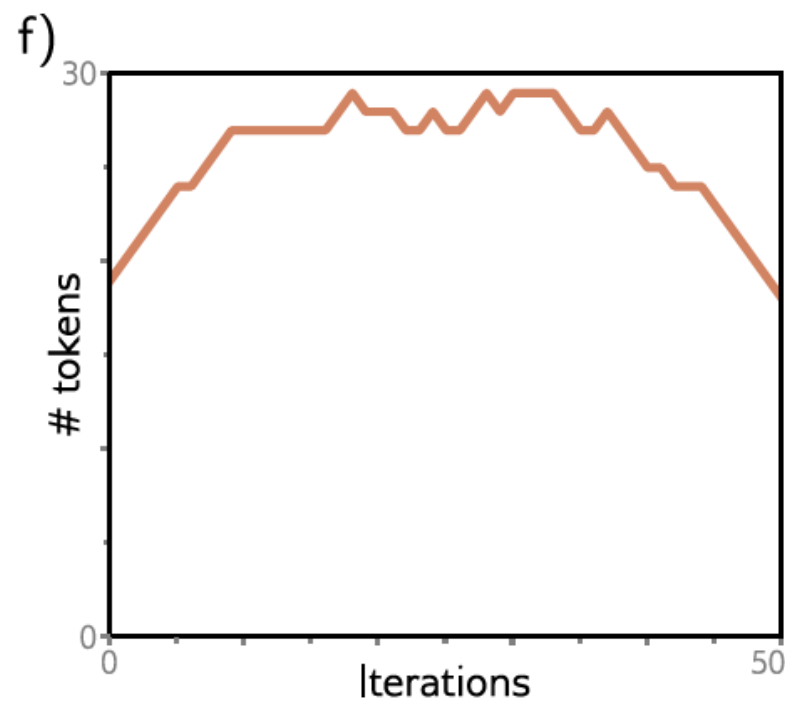
_	se	a	e	t	o	h	l	u	b	d	e	w	c	s	f	i	m	n	p	r
21	13	12	12	11	8	6	6	4	3	3	3	3	2	2	1	1	1	1	1	1

⋮ ⋮

d) see_sea_e_b_l_w_a_could_hat_he_o_t_t_the_to_u_a_d_f_m_n_p_s_sailor_to
 7 6 4 3 3 3 3 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1

⋮ ⋮ ⋮

e) see_sea_could_he_the_a_all_blue_bottom_but_deep_of_sailor_that_to_was_went_what_
 7 6 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

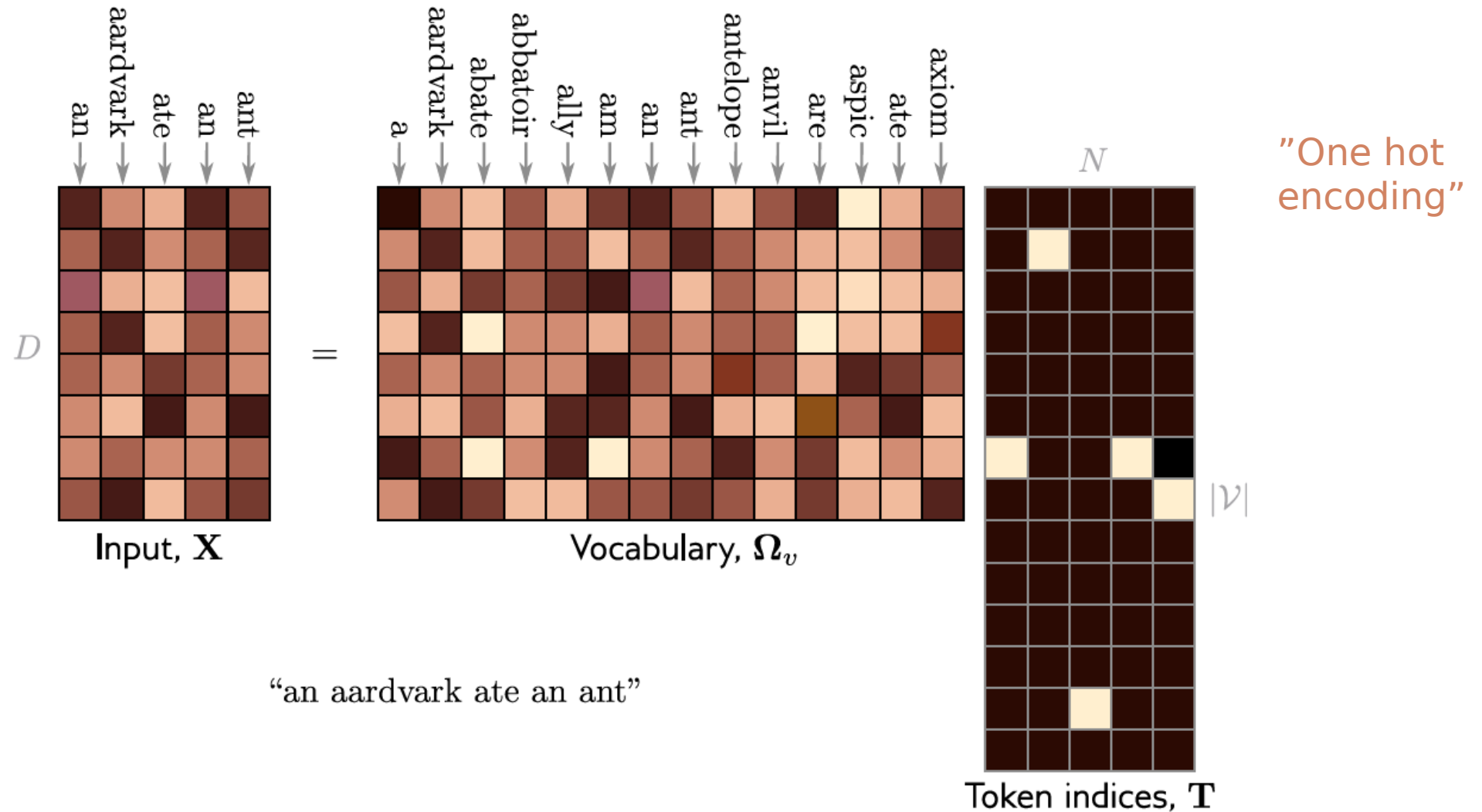


- First, number of tokens increases, as in addition to letters combinations become tokens.
- Later the fragments are merged, decreasing the number of tokens again.

Embeddings

- Each token is mapped to a unique word embedding.
- Embeddings Ω_e for a whole vocabulary are stored in a matrix.
- Input embeddings: $\mathbf{X} = \Omega_e \mathbf{T}$
- Ω_e is learned

Learning vocabulary



Transformer Model

- The embedding matrix **X** is passed through a series of K transformer layers.
- **Encoder** transforms the text embeddings into a **representation** that can **support a variety of tasks** (e.g. text classification. BERT).
- **Decoder** predicts next token to continue the input. (e.g. completing text; story telling. GPT-3)
- **Encoder-decoders** are used in sequence to sequence tasks (e.g. machine translation)